MLGWorks DevConsole

Modular developer console for Unity

Setup, configuration, extension, and production guide

MLGWorks

Version 1.0.0 • August 18, 2026

Contents

1 Overview

MLGWorks DevConsole is a runtime developer console for Unity projects. It provides a visible command interface for development workflows while keeping command registration explicit, configurable, and testable. The system includes command execution, autocomplete, command history, scrollback, Unity logging integration, and an editor-managed command catalog.

The main design principle is that discovery happens in the editor and execution happens from a serialized catalog. This makes the set of available commands reviewable in source control and prevents an accidental runtime reflection scan from exposing commands in a build.

Important

The catalog is the source of truth at runtime. A missing catalog is a configuration error and results in zero registered commands.

2 Requirements and installation

2.1 Requirements

- Unity 6000.0 or newer.
- The Unity Input System package.
- The MLGWorks.Utils dependency.

2.2 Installing from a Unity project

For a Git-based UPM installation, add the package dependency in the consuming project's `Packages/manifest.json`:

```
"com.mlgworks.devconsole":
  "https://github.com/TrickShotMLG02/MLGWorks.DevConsole.git"
```

If the repository is being used directly as an Assets package, place the `MLGWorks` directory under the project's `Assets` directory and allow Unity to import the generated metadata files.

2.3 First-time setup

1. Open the project in Unity.
2. Add the `DevConsole` prefab to a scene.
3. Open **Window** → **MLGWorks** → **DevConsole Commands**.
4. Create a `DevConsoleCommandSettings` asset, or select the included asset.
5. Click **Refresh Discovery**.
6. Assign the catalog asset to the selected DevConsole object. The included sample scene already references `Samples/DevConsoleSample/DevConsoleCommandSettings.asset`.
7. Enter Play Mode and use the configured input action to open the console.

3 Architecture

The runtime is split into focused services. `DevConsole` coordinates the services, while interfaces make the core behavior easy to replace in tests or custom integrations.

Service	Responsibility
`ICommandRegistry`	Stores command metadata and registers commands from the catalog.
`ICommandExecutor`	Parses input, converts arguments, and invokes registered commands.
`IAutocompleteEngine`	Provides command and argument completion behavior.
`ICommandHistory`	Tracks previous input and draft restoration.
`IConsoleInput`	Abstracts input action subscription and disposal.
`IConsoleOutput`	Receives command results and log messages.
`IScrollbackBuffer`	Limits and stores displayed console lines.

4 Defining commands

Commands are public or private static methods decorated with `CommandAttribute`. The method's declaring type does not need to inherit from a special base class.

```
using MLGWorks.DevConsole.Runtime.Commands;

public static class GameCommands
{
    [Command("weather", "Changes the current weather", "w")]
    public static string Weather(string value)
    {
        return $"Weather changed to {value}.";
    }
}
```

The first constructor argument is the command name. The second is an optional description. Remaining positional string arguments are aliases.

4.1 Parameters

Parameters are parsed from whitespace-separated input. Supported conversions include strings, booleans, numeric types, enums, and types handled by the utility conversion layer. Optional C# parameters are represented as optional command arguments.

```
[Command("spawn", "Spawns an object")]
public static string Spawn(string prefab, int count = 1)
{
    return $"Requested {count} instance(s) of {prefab}.";
}
```

4.2 Danger levels and safe defaults

Commands can declare their editor presentation and initial catalog state with named attribute properties:

```
[Command(
    "reset-save",
```

```
    "Deletes the current save data",
    DangerLevel = CommandDangerLevel.Dangerous,
    EnabledByDefault = false)]
public static string ResetSave() => "Save reset.";
```

The available levels are `None`, `Warning`, and `Dangerous`. The catalog displays warnings in yellow and dangerous commands in red. A category uses the highest severity among its commands. `EnabledByDefault` only controls a newly discovered entry; rediscovery preserves an existing user's enabled state.

Safety note

Danger levels are a review and UX aid, not an authorization system. Add project-specific confirmation, permissions, or build constraints before exposing destructive commands to untrusted users.

5 Command catalog workflow

5.1 Discovery

The editor scans loaded assemblies for methods carrying `CommandAttribute`. Each entry stores the assembly, declaring type, method, parameter type names, command name, aliases, description, danger level, and state. A stable identity is derived from the assembly, type, method, and parameter signature.

5.2 Refreshing without losing settings

Click **Refresh Discovery** after adding or changing command methods. Existing entries are matched by stable identity, so their enabled or disabled state is retained. New entries use `EnabledByDefault`.

5.3 Classes and bulk controls

Commands are grouped by declaring type. The class checkbox disables or enables the complete class. Each category also has one dynamic bulk button: it shows **Enable All** when an eligible command is disabled and **Disable All** when all eligible commands are enabled.

5.4 Obsolete entries

When a command disappears from source, the catalog retains it as obsolete. Obsolete entries are not registered at runtime. This allows accidentally deleted scripts to be restored without losing their previous state. Use **Remove Obsolete** only after confirming that recovery is no longer needed.

5.5 Test-only entries

Commands discovered in assemblies named like `.Tests`, `.EditorTests`, or `.PlayModeTests` are marked test-only. They remain visible for review but are disabled in the catalog, excluded from runtime registration, and excluded from command/disabled metrics. Tests can still call methods directly or explicitly use the test registry path.

5.6 Search

The command window supports focused search:

- No prefix: search command names only.
- `$`: search declaring classes and namespaces.
- `#`: search all catalog metadata, including command, class, namespace, assembly, and method names.

6 Runtime behavior

At startup, DevConsole first uses an explicitly assigned settings asset. If none is assigned, it loads `DevConsoleCommandSettings` from the Resources folder. If neither is available, it clears the command registry and logs a configuration warning. It never silently falls back to runtime reflection discovery.

```
// Explicit assignment is preferred on the DevConsole component.
// Otherwise place the catalog at:
// Assets/MLGWorks/Samples/DevConsoleSample/DevConsoleCommandSettings.asset
```

Only enabled, non-obsolete, non-test-only commands from enabled classes are resolved and registered. Aliases are registered alongside the primary name. Invalid or stale entries are skipped with a warning.

7 Built-in commands

The package includes several command groups:

Group	Purpose
`BaseCommands`	Help, clear, and console control commands.
`MathCommands`	Basic arithmetic, rounding, comparison, and numeric helpers.
`VariablesCommands`	Reflection-based get/set access to fields and properties.
`ExecutionCommands`	Reflection-based method invocation; disabled by default and dangerous.

The `get` command is a warning-level command. The `set` and `invoke` commands are dangerous; `invoke` is disabled by default.

8 Input, UI, and logging

The prefab includes the input and UI components required to display the console. The Input System action asset provides the open/close, submit, navigation, and autocomplete actions. The input source implements disposal so action maps are disabled and released with the component lifecycle.

The console subscribes to the MLGWorks logging service and forwards new log batches into a bounded scrollback buffer. Configure the UI references on the prefab if creating a custom console layout.

9 Testing

Run both test suites in Unity Test Runner:

1. Open **Window** → **General** → **Test Runner**.
2. Run the **EditMode** tests.
3. Run the **PlayMode** tests.
4. Test a player build after changing command signatures or catalog entries.

The tests cover core services, command parsing, history edge cases, autocomplete, reflection helpers, built-in commands, math edge cases, catalog state preservation, obsolete commands, test-only filtering, input disposal, and regression scenarios.

10 Troubleshooting

Symptom	Resolution
No commands appear	Assign a catalog or put it under a Resources folder, then refresh discovery.
A command is missing	Check the command and declaring class are enabled, then inspect obsolete entries and refresh.
A command is stale	The method signature changed; refresh discovery and review the obsolete entry.
Test commands appear disabled	This is intentional; they are excluded from runtime catalog registration.
Danger color is missing	Refresh discovery so the catalog receives current attribute metadata.
A player build cannot resolve a command	Add preservation metadata for reflection and verify the method is included in the target assembly.

11 Production checklist

- Refresh and review the command catalog before creating a release build.
- Remove obsolete entries only after confirming they are no longer needed.
- Keep dangerous commands disabled unless the build explicitly requires them.
- Add confirmation and authorization around destructive operations.
- Verify the catalog is included in the target build.
- Run EditMode and PlayMode tests.
- Test an IL2CPP player build for reflection and code stripping issues.
- Review the catalog and generated build contents before distribution.

12 Extending the asset

New commands require only the attribute and a compatible static method. For larger integrations, depend on the public interfaces rather than concrete implementations. This allows custom input sources, output sinks, command registries, autocomplete providers, and history implementations without changing the console's orchestration code.

When adding new serialized catalog metadata, preserve stable identities and add migration tests. Never overwrite existing user state during discovery unless the change is explicitly documented as breaking.

13 License

MLGWorks DevConsole is distributed under the MIT License. See the repository's `LICENSE` file for the complete text.